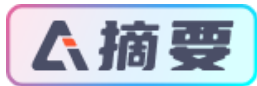


FreeRTOS事件组 基于STM32



本文详细介绍了FreeRTOS中事件组的概念、应用场景、运作机制及关键API的使用方法，包括事件的创建、删除、置位、等待和清除等操作。

概述

文章对事件组的，应用场景，运作机制，以及事件的创建，删除，等待，置位，同步等操作

文章目录

[概述](#)

[一、事件标志组简介](#)

[1、事件位\(事件标志\)](#)

[2、事件组](#)

[3、事件标志组和事件位的数据类型](#)

[二、事件的应用场景](#)

[三、事件运作机制](#)

[四、事件控制块](#)

[五、事件组函数](#)

[1.事件创建函数 xEventGroupCreate\(\)](#)

[2.事件删除函数 vEventGroupDelete\(\)](#)

[3.事件组置位函数 xEventGroupSetBits\(\)_\(任务\)](#)

[4.等待事件函数 xEventGroupWaitBits\(\)](#)

[5.xEventGroupClearBits\(\)与 xEventGroupClearBitsFromISR\(\)](#)

[六、事件实验](#)

[七、实验现象](#)

一、事件标志组简介

1、事件位(事件标志)

事件位用来表明某个事件是否发生，事件位通常用作事件标志，比如下面的几个例子：

- 当收到一条消息并且把这条消息处理掉以后就可以将某个位(标志)置 1，当队列中没有 消息需要处理的时候就可以将这个位(标志)置 0。
- 当把队列中的消息通过网络发送输出以后就可以将某个位(标志)置 1，当没有数据需要 从网络发送出去的话就将这个位(标志)置 0。

● 现在需要向网络中发送一个心跳信息，将某个位(标志)置 1。现在不需要向网络中发送心跳信息，这个位(标志)置 0。

2、事件组

一个事件组就是一组的事件位，事件组中的事件位通过位编号来访问，同样，以上面列出的三个例子为例：

- 事件标志组的 bit0 表示队列中的消息是否处理掉。
- 事件标志组的 bit1 表示是否有消息需要从网络中发送出去。
- 事件标志组的 bit2 表示现在是否需要向网络发送心跳信息。

3、事件标志组和事件位的数据类型

事件标志组的数据类型为 EventGroupHandle_t，当 configUSE_16_BIT_TICKS 为 1 的时候事件标志组可以存储 8 个事件位，当 configUSE_16_BIT_TICKS 为 0 的时候事件标志组存储 24 个事件位。

事件标志组中的所有事件位都存储在一个无符号的 EventBits_t 类型的变量中，EventBits_t 在 event_groups.h 中有如下定义：

```
typedef TickType_t EventBits_t;
```

数据类型 TickType_t 在文件 portmacro.h 中有如下定义：

```
1. #if( configUSE_16_BIT_TICKS == 1 )
2.     typedef uint16_t TickType_t;
3. #define portMAX_DELAY ( TickType_t ) 0xffff
4. #else
5.     typedef uint32_t TickType_t;
6. #define portMAX_DELAY ( TickType_t ) 0xffffffffUL
7. #define portTICK_TYPE_IS_ATOMIC 1
8. #endif
```

可以看出当 configUSE_16_BIT_TICKS 为 0 的时候 TickType_t 是个 32 位的数据类型，因此 EventBits_t 也是个 32 位的数据类型。EventBits_t 类型的变量可以存储 24 个事件位，另外的那高 8 位有其他用。事件位 0 存放在这个变量的 bit0 上，变量的 bit1 就是事件位 1，以此类推。对于 STM32 来说一个事件标志组最多可以存储 24 个事件位，如图1所示

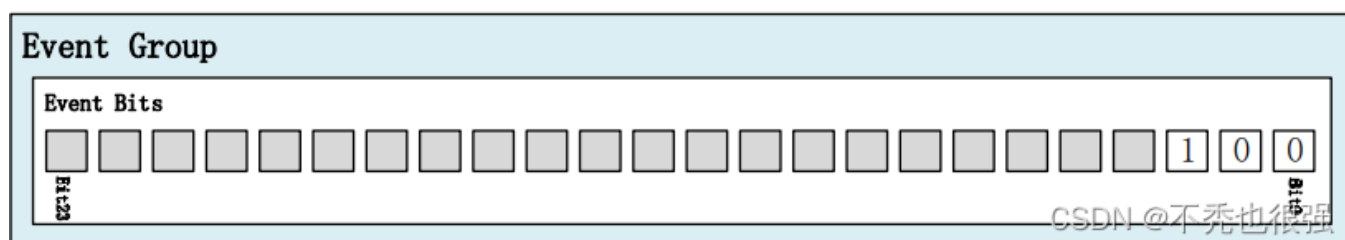


图1 事件标志组和事件位

二、事件的应用场景

FreeRTOS 的事件用于事件类型的通讯，无数据传输，也就是说，我们可以用事件来做标志位，判断某些事件是否发生了，然后根据结果做处理，那很多人又会问了，为什么我不直接用变量做标志呢，岂不是更好更有效率？非也非也，若是在裸机编程中，用全局变量是最为有效的方法，这点我不否认，但是在操作系统中，使用全局变量就要考虑以下问题了：

如何对全局变量进行保护呢，如何处理多任务同时对它进行访问？

如何让内核对事件进行有效管理呢？使用全局变量的话，就需要在任务中轮询查看事件是否发送，这简直就是在浪费 CPU 资源啊，还有等待超时机制，使用全局变量的话需要用户自己去实现。

所以，在操作系统中，还是使用操作系统给我们提供的通信机制就好了，简单方便还实用。

在某些场合，可能需要多个时间发生了才能进行下一步操作，比如一些危险机器的启动，需要检查各项指标，当指标不达标的时候，无法启动，但是检查各个指标的时候，不能一下子检测完毕啊，所以，需要事件来做统一的等待，当所有的事件都完成了，那么机器才允许启动，这只是事件的其中一个应用。

事件可用于多种场合，它能够在一定程度上替代信号量，用于任务与任务间，中断与任务间的同步。一个任务或中断服务例程发送一个事件给事件对象，而后等待的任务被唤醒并对相应的事件进行处理。但是它与信号量不同的是，事件的发送操作是不可累计的，而信号量的释放动作是可累计的。事件另外一个特性是，接收任务可等待多种事件，即多个事件对应一个任务或多个任务。同时按照任务等待的参数，可选择是“逻辑或”触发还是“逻辑与”触发。这个特性也是信号量等所不具备的，信号量只能识别单一同步动作，而不能同时等待多个事件的同步。

各个事件可分别发送或一起发送给事件对象，而任务可以等待多个事件，任务仅对感兴趣的事件进行关注。当有它们感兴趣的事件发生时并且符合感兴趣的条件，任务将被唤醒并进行后续的处理动作。

三、事件运作机制

接收事件时，可以根据感兴趣的参事件类型接收事件的单个或者多个事件类型。事件接收成功后，必须使用 `xClearOnExit` 选项来清除已接收到的事件类型，否则不会清除已接收到的事件，这样就需要用户显式清除事件。

用户可以自定义通过传入参数 `xWaitForAllBits` 选择读取模式，是等待所有感兴趣的事件还是等待感兴趣的任意一个事件。

设置事件时，对指定事件写入指定的事件类型，设置事件集合的对应事件位为 1，可以一次同时写多个事件类型，设置事件成功可能会触发任务调度。

清除事件时，根据入参数事件句柄和待清除的事件类型，对事件对应位进行清 0 操作。事件不与任务相关联，事件相互独立，一个 32 位的变量（事件集合，实际用于表示事件的只有 24 位），用于标识该任务发生的事件类型，其中每一位表示一种事件类型（0 表示该事件类型未发生、1 表示该事件类型已经发生），一共 24 种事件类型具体见图 2

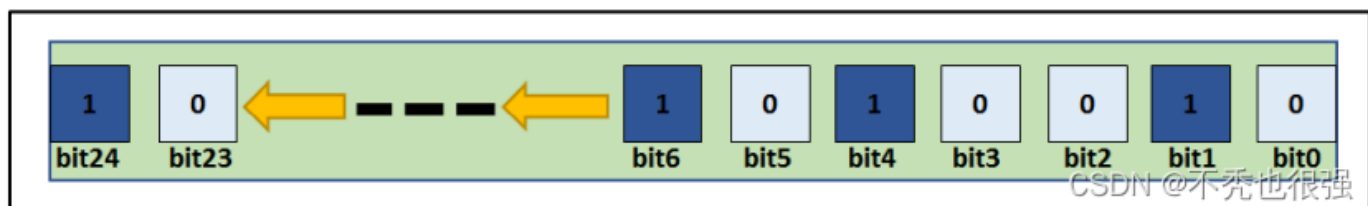


图2事件集合 set（一个 32 位的变量）

事件唤醒机制，当任务因为等待某个或者多个事件发生而进入阻塞态，当事件发生的时候会被唤醒，其过程具体见图 3

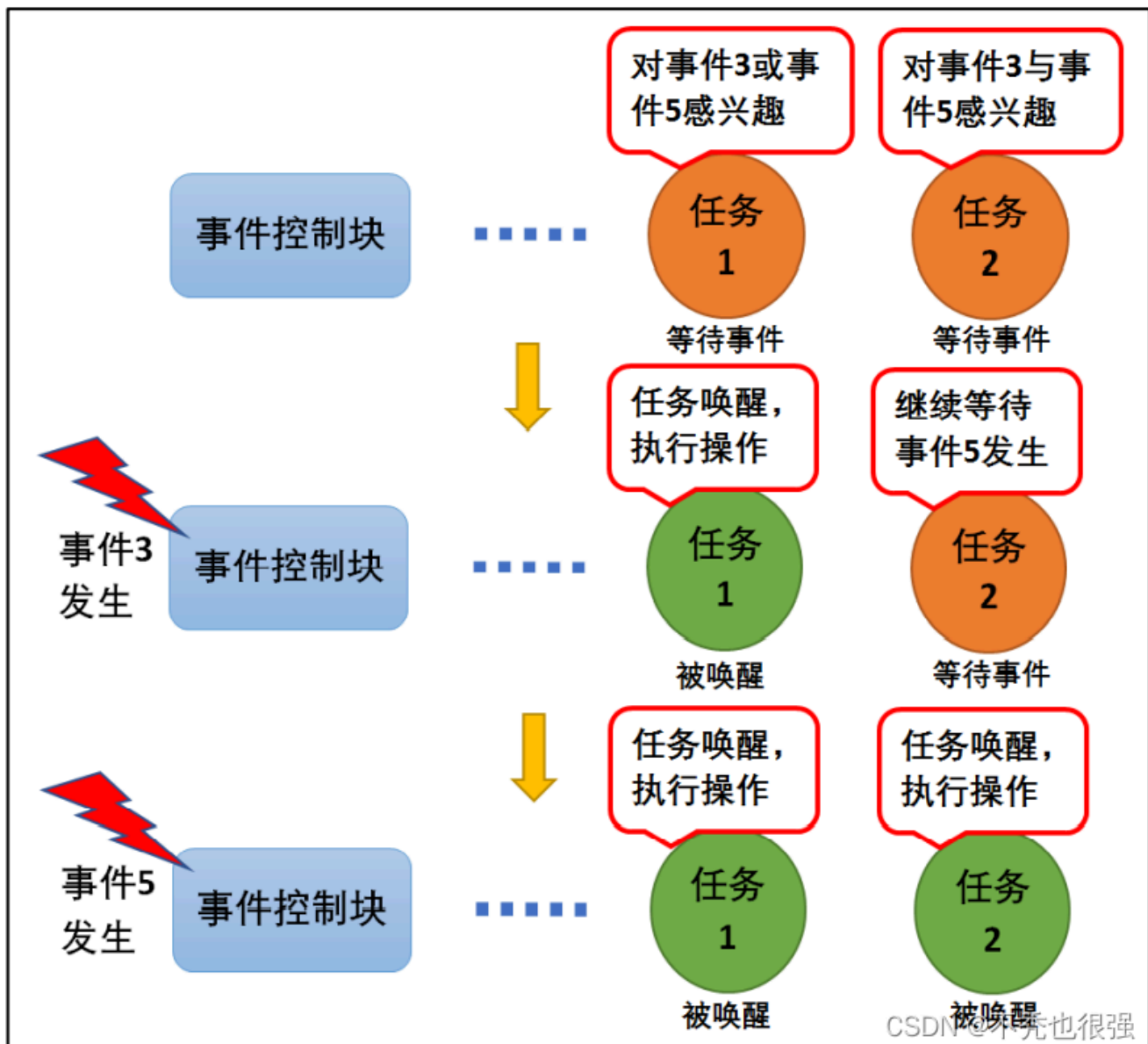


图3 事件唤醒任务示意图

任务 1 对事件 3 或事件 5 感兴趣（逻辑或），当发生其中的某一个事件都会被唤醒，并且执行相应操作。而任务 2 对事件 3 与事件 5 感兴趣（逻辑与），当且仅当事件 3 与事件 5 都发生的时候，任务 2 才会被唤醒，如果只有一个其中一个事件发生，那么任务还是会继续等待事件发生。如果接在收事件函数中设置了清除事件位 `xClearOnExit`，那么当任务唤醒后将把事件 3 和事件 5 的事件标志清零，否则事件标志将依然存在。

四、事件控制块

事件标志组存储在一个 `EventBits_t` 类型的变量中，该变量在事件组结构体中定义，具体见代码清单 20-1 加粗部分。如果宏 `configUSE_16_BIT_TICKS` 定义为 1，那么变量 `uxEventBits` 就是 16 位的，其中有 8 个位用来存储事件组，如果宏 `configUSE_16_BIT_TICKS` 定义为 0，那么变量 `uxEventBits` 就是 32 位的，其中有 24 个位用来存储事件组，每一位代表一个事件的发生与否，利用逻辑或、逻辑与等实现不同事件的不同唤醒处理。在 STM32 中，`uxEventBits` 是 32 位的，所以我们有 24 个位用来实现事件组。除了事件标志组变量之外，FreeRTOS 还使用了一个链表来记录等待事件的任务，所有在等待此事件的任务均会被挂载在等待事件列表 `xTasksWaitingForBits`。

```

1. typedef struct xEventGroupDefinition {
2.     EventBits_t uxEventBits;
3.     List_t xTasksWaitingForBits;
4.
5.     #if( configUSE_TRACE_FACILITY == 1 )
6.         UBaseType_t uxEventGroupNumber;
7.     #endif
8.
9.     #if( ( configSUPPORT_STATIC_ALLOCATION == 1 ) \
10.         && ( configSUPPORT_DYNAMIC_ALLOCATION == 1 ) )

```

```
11.     uint8_t ucStaticallyAllocated;
12. #endif
13.     } EventGroup_t;
```

五、事件组函数

1.事件创建函数 xEventGroupCreate()

事件创建函数，顾名思义，就是创建一个事件，与其他内核对象一样，都是需要先创建才能使用的资源，FreeRTOS 给我们提供了一个创建事件的函数 xEventGroupCreate()，当创建一个事件时，系统会首先给我们分配事件控制块的内存空间，然后对该事件控制块进行基本的初始化，创建成功返回事件句柄；创建失败返回 NULL。所以，在使用创建函数之前，我们需要先定义有个事件的句柄。

```
1. /* 创建一个事件组，返回它的句柄。
2.  * 此函数内部会分配事件组结构体
3.  * 返回值：返回句柄，非NULL表示成功
4.  */
5. EventGroupHandle_t xEventGroupCreate( void );
6.
7. /* 创建一个事件组，返回它的句柄。
8.  * 此函数无需动态分配内存，所以需要先有一个StaticEventGroup_t结构体，并传入它的指针
9.  * 返回值：返回句柄，非NULL表示成功
10. */
11. EventGroupHandle_t xEventGroupCreateStatic( StaticEventGroup_t * pxEventGroupBuffer );
```

2.事件删除函数 vEventGroupDelete()

vEventGroupDelete()用于删除由函数 xEventGroupCreate()创建的事件组，只有被创建成功的事件才能被删除，但是需要注意的是该函数不允许在中断里面使用。当事件组被删除之后，阻塞在该事件组上的任务都会被解锁，并向等待事件的任务返回事件组的值为 0，其使用是非常简单的。

```
1. /*
2.  * xEventGroup: 事件组句柄，你要删除哪个事件组
3.  */
4. void vEventGroupDelete( EventGroupHandle_t xEventGroup )
```

3.事件组置位函数 xEventGroupSetBits()（任务）

xEventGroupSetBits()用于置位事件组中指定的位，当位被置位之后，阻塞在该位上的任务将会被解锁。使用该函数接口时，通过参数指定的事件标志来设定事件的标志位，然后遍历等待在事件对象上的事件等待列表，判断是否有任务的事件激活要求与当前事件对象标志值匹配，如果有，则唤醒该任务。简单来说，就是设置我们自己定义的事件标志位为 1，并且看看有没有任务在等待这个事件，有的话就唤醒它。

xEventGroupSetBits()函数说明

函数	EventBits_t xEventGroupSetBits(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToSet); CSDN @不秃也很强	
原型		
功能	置位事件组中指定的位。	
参数	xEventGroup	事件句柄。
	uxBitsToSet	指定事件中的事件标志位。如设置 uxBitsToSet 为 0x08 则只置位位 3，如果设置 uxBitsToSet 为 0x09 则位 3 和位 0 都需要被置位。
返回值	返回调用 xEventGroupSetBits() 时事件组中的值。 CSDN @不秃也很强	

xEventGroupSetBits()函数使用实例

```
1. #define KEY1_EVENT  (0x01 << 0)//设置事件掩码的位0
2. #define KEY2_EVENT  (0x01 << 1)//设置事件掩码的位1
3. static EventGroupHandle_t Event_Handle =NULL;
4. Event_Handle = xEventGroupCreate();
5. if(NULL != Event_Handle)
6.     printf("Event_Handle 事件创建成功!\r\n");
7. static void KEY_Task(void* parameter)
8. {
9.     /* 任务都是一个无限循环，不能返回 */
10.    while (1)
11.    {
12.        if( Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN) == KEY_ON )           //如果KEY2被单击
13.        {
14.            printf ( "KEY1被按下\n" );
15.            /* 触发一个事件1 */
16.            xEventGroupSetBits(Event_Handle,KEY1_EVENT);
17.        }
18.
19.        if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN) == KEY_ON )           //如果KEY2被单击
20.        {
21.            printf ( "KEY2被按下\n" );
22.            /* 触发一个事件2 */
23.            xEventGroupSetBits(Event_Handle,KEY2_EVENT);
24.        }
25.        vTaskDelay(20);           //每20ms扫描一次
26.    }
27. }
```



4.等待事件函数 xEventGroupWaitBits()

既然标记了事件的发生，那么我怎么知道他到底有没有发生，这也是需要一个函数来获取事件是否已经发生，FreeRTOS 提供了一个等待指定事件的函数——xEventGroupWaitBits()，通过这个函数，任务可以知道事件标志组中的哪些位，有什么事件发生了，然后通过“逻辑与”、“逻辑或”等操作对感兴趣的事件进行获取，并且这个函数实现了等待超时机制，当且仅当任务等待的事件发生时，任务才能获取到事件信息。在这段时间中，如果事件一直没发生，该任务将保持阻塞状态以等待事件发生。当其它任务或中断服务程序往其等待的事件设置对应的标志位，该任务将自动由阻塞态转为就绪态。当任务等待的时间超过了指定的阻塞时间，即使事件还未发生，任务也会自动从阻塞态转移为就绪态。这样子很有效的体现了操作系统的实时性，如果事件正确获取（等待到）则返回对应的事件标志位，由用户判断再做处理，因为在事件超时的时候也会返回一个不能确定的事件值，所以需要判断任务所等待的事件是否真的发生。

xEventGroupWaitBits()函数说明

函数原型	EventBits_t xEventGroupWaitBits(const EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToWaitFor, const BaseType_t xClearOnExit, const BaseType_t xWaitForAllBits, TickType_t xTicksToWait);	
功能	用于获取任务感兴趣的事件。	
参数	xEventGroup	事件句柄。
	uxBitsToWaitFor	一个按位或的值，指定需要等待事件组中的哪些位置 1。如果需要等待 bit 0 and/or bit 2 那么 uxBitsToWaitFor 配置为 0x05(0101b)。如果需要等待 bits 0 and/or bit 1 and/or bit 2 那么 uxBitsToWaitFor 配置为 0x07(0111b)。
	xClearOnExit	pdTRUE：当 xEventGroupWaitBits() 等待到满足任务唤醒的事件时，系统将清除由形参 uxBitsToWaitFor 指定的事件标志位。 pdFALSE：不会清除由形参 uxBitsToWaitFor 指定的事件标志位。
	xWaitForAllBits	pdTRUE：当形参 uxBitsToWaitFor 指定的位都置位的时候，xEventGroupWaitBits()才满足任务唤醒的条件，这也是“逻辑与”等待事件，并且在没有超时的情况下返回对应的事件标志位的值。 pdFALSE：当形参 uxBitsToWaitFor 指定的位有其中任意一个置位的时候，这也是常说的“逻辑或”等待事件，在没有超时的情况下函数返回对应的事件标志位的值。
	xTicksToWait	最大超时时间，单位为系统节拍周期，常量 portTICK_PERIOD_MS 用于辅助把时间转换成 MS。
返回值	返回事件中的哪些事件标志位被置位，返回值很可能并不是用户指定的事件位，需要对返回值进行判断再处理。	

CSDN @不秃也很强

CSDN @不秃也很强

xEventGroupWaitBits()使用实例

```

1. static void LED_Task(void* parameter)
2. {
3.     EventBits_t r_event; /* 定义一个事件接收变量 */
4.     /* 任务都是一个无限循环，不能返回 */
5.     while (1)
6.     {
7.         /*****
8.          * 等待接收事件标志
9.          *
10.         * 如果xClearOnExit设置为pdTRUE，那么在xEventGroupWaitBits()返回之前，
11.         * 如果满足等待条件（如果函数返回的原因不是超时），那么在事件组中设置
12.         * 的uxBitsToWaitFor中的任何位都将被清除。
13.         * 如果xClearOnExit设置为pdFALSE，
14.         * 则在调用xEventGroupWaitBits()时，不会更改事件组中设置的位。
15.         *
16.         * xWaitForAllBits如果xWaitForAllBits设置为pdTRUE，则当uxBitsToWaitFor中
17.         * 的所有位都设置或指定的块时间到期时，xEventGroupWaitBits()才返回。
18.         * 如果xWaitForAllBits设置为pdFALSE，则当设置uxBitsToWaitFor中设置的任何
19.         * 一个位置1 或指定的块时间到期时，xEventGroupWaitBits()都会返回。
20.         * 阻塞时间由xTicksToWait参数指定。
21.         *****/
22.         r_event = xEventGroupWaitBits(Event_Handle, /* 事件对象句柄 */
23.                                     KEY1_EVENT|KEY2_EVENT, /* 接收线程感兴趣的事件 */
24.                                     pdTRUE, /* 退出时清除事件位 */
25.                                     pdTRUE, /* 满足感兴趣的所有事件 */
26.                                     portMAX_DELAY); /* 指定超时事件，一直等 */
27.
28.         if((r_event & (KEY1_EVENT|KEY2_EVENT)) == (KEY1_EVENT|KEY2_EVENT))
29.         {
30.             /* 如果接收完成并且正确 */
31.             printf ("KEY1与KEY2都按下\n");

```

```
32.     LED1_TOGGLE;          //LED1 反转
33. }
34. else
35.     printf ( "事件错误! \n");
36. }
37. }
```

5. xEventGroupClearBits()与 xEventGroupClearBitsFromISR()

xEventGroupClearBits()与 xEventGroupClearBitsFromISR()都是用于清除事件组指定的 位，如果在获取事件的时候没有将对应的标志位清除，那么就需要用这个函数来进行显式 清除，xEventGroupClearBits()函数不能在中断中使用，而是由具有中断保护功能的 xEventGroupClearBitsFromISR() 来代替，中断清除事件标志位的操作在守护任务（也叫定时器服务任务）里面完成。守护进程的优先级由FreeRTOSConfig.h中的宏 configTIMER_TASK_PRIORITY 来定义。要想使用该函数必须把FreeRTOS/source/event_groups.c 这个 C 文件添加到工程中。

xEventGroupClearBits()与 xEventGroupClearBitsFromISR()函数说明

函数原型	EventBits_t xEventGroupClearBits(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToClear); BaseType_t xEventGroupClearBitsFromISR(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToClear);	
功能	清除事件组中指定的位。	
参数	xEventGroup	事件句柄。
	uxBitsToClear	指定事件组中的哪个位需要清除。如设置 uxBitsToSet 为 0x08 则只清除位 3，如果设置 uxBitsToSet 为 0x09 则位 3 和位 0 都需要被清除。
返回值	事件在还没有清除指定位之前的值。	

CSDN @不秃也很强

xEventGroupClearBits()函数使用实例

```
1.  #define BIT_0 ( 1 << 0 )
2.  #define BIT_4 ( 1 << 4 )
3.
4.  void aFunction( EventGroupHandle_t xEventGroup )
5.  {
6.      EventBits_t uxBits;
7.
8.      /* 清楚事件组的 bit 0 and bit 4 */
9.      uxBits = xEventGroupClearBits(
10.         xEventGroup,
11.         BIT_0 | BIT_4 );
12.
13.      if ( ( uxBits & ( BIT_0 | BIT_4 ) ) == ( BIT_0 | BIT_4 ) ) {
14.          /* 在调用 xEventGroupClearBits()之前 bit0 和 bit4 都置位
15.             但是现在是被清除了*/
16.      } else if ( ( uxBits & BIT_0 ) != 0 ) {
17.          /* 在调用 xEventGroupClearBits()之前 bit0 已经置位
18.             但是现在是被清除了*/
19.      } else if ( ( uxBits & BIT_4 ) != 0 ) {
20.          /* 在调用 xEventGroupClearBits()之前 bit4 已经置位
21.             但是现在是被清除了*/
22.      } else {
23.          /* 在调用 xEventGroupClearBits()之前 bit0 和 bit4 都没被置位 */
24.      }
25.  }
```


六、事件实验

事件标志组实验是在 FreeRTOS 中创建了两个任务，一个是设置事件任务，一个是等待事件任务，两个任务独立运行，设置事件任务通过检测按键的按下情况设置不同的事件标志位，等待事件任务则获取这两个事件标志位，并且判断两个事件是否都发生，如果是则输出相应信息，LED 进行翻转。等待事件任务的等待时间是 portMAX_DELAY，一直在等待事件的发生，等待到事件之后清除对应的事件标记位，具体见代码清单 20-13 加粗部分。

```
1.
2. /* FreeRTOS头文件 */
3. #include "FreeRTOS.h"
4. #include "task.h"
5. #include "event_groups.h"
6. /* 开发板硬件bsp头文件 */
7. #include "bsp_led.h"
8. #include "bsp_usart.h"
9. #include "bsp_key.h"
10. /***** 任务句柄 *****/
11. /*
12. * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
13. * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
14. * 这个句柄可以为NULL。
15. */
16. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
17. static TaskHandle_t LED_Task_Handle = NULL; /* LED_Task任务句柄 */
18. static TaskHandle_t KEY_Task_Handle = NULL; /* KEY_Task任务句柄 */
19.
20. /***** 内核对象句柄 *****/
21. /*
22. * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
23. * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
24. * 们就可以通过这个句柄操作这些内核对象。
25. *
26. * 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
27. * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
28. * 来完成的
29. *
30. */
31. static EventGroupHandle_t Event_Handle = NULL;
32.
33. /***** 全局变量声明 *****/
34. /*
35. * 当我们在写应用程序的时候，可能需要用到一些全局变量。
36. */
37.
38.
39. /***** 宏定义 *****/
40. /*
41. * 当我们在写应用程序的时候，可能需要用到一些宏定义。
42. */
43. #define KEY1_EVENT (0x01 << 0) //设置事件掩码的位0
44. #define KEY2_EVENT (0x01 << 1) //设置事件掩码的位1
45.
46. /*
47. *****/
48. * 函数声明
49. *****/
50. */
51. static void AppTaskCreate(void); /* 用于创建任务 */
52.
53. static void LED_Task(void* pvParameters); /* LED_Task 任务实现 */
54. static void KEY_Task(void* pvParameters); /* KEY_Task 任务实现 */
55.
56. static void BSP_Init(void); /* 用于初始化板载相关资源 */
57.
58. /*****
59. * @brief 主函数
60. * @param 无
61. * @retval 无
62. * @note 第一步：开发板硬件初始化
63. 第二步：创建APP应用任务
64. 第三步：启动FreeRTOS，开始多任务调度
65. *****/
66. int main(void)
67. {
68. BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
69.
70. /* 开发板硬件初始化 */
71. BSP_Init();
72. printf("这是一个FreeRTOS事件标志组实验！\n");
73. /* 创建AppTaskCreate任务 */
```

```

74.   xReturn = xTaskCreate((TaskFunction_t)AppTaskCreate, /* 任务入口函数 */
75.                       (const char*   )"AppTaskCreate", /* 任务名字 */
76.                       (uint16_t      )512, /* 任务栈大小 */
77.                       (void*         )NULL, /* 任务入口函数参数 */
78.                       (UBaseType_t   )1, /* 任务的优先级 */
79.                       (TaskHandle_t*  )&AppTaskCreate_Handle); /* 任务控制块指针 */
80.   /* 启动任务调度 */
81.   if(pdPASS == xReturn)
82.       vTaskStartScheduler(); /* 启动任务，开启调度 */
83.   else
84.       return -1;
85.
86.   while(1); /* 正常不会执行到这里 */
87. }
88.
89.
90. /*****
91.  * @ 函数名 : AppTaskCreate
92.  * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
93.  * @ 参数 : 无
94.  * @ 返回值 : 无
95.  *****/
96. static void AppTaskCreate(void)
97. {
98.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
99.
100.    taskENTER_CRITICAL(); /* 进入临界区
101.
102.    /* 创建 Event_Handle */
103.    Event_Handle = xEventGroupCreate();
104.    if(NULL != Event_Handle)
105.        printf("Event_Handle 事件创建成功!\r\n");
106.
107.    /* 创建LED_Task任务 */
108.    xReturn = xTaskCreate((TaskFunction_t)LED_Task, /* 任务入口函数 */
109.                        (const char*   )"LED_Task", /* 任务名字 */
110.                        (uint16_t      )512, /* 任务栈大小 */
111.                        (void*         )NULL, /* 任务入口函数参数 */
112.                        (UBaseType_t   )2, /* 任务的优先级 */
113.                        (TaskHandle_t*  )&LED_Task_Handle); /* 任务控制块指针 */
114.    if(pdPASS == xReturn)
115.        printf("创建LED_Task任务成功!\r\n");
116.
117.    /* 创建KEY_Task任务 */
118.    xReturn = xTaskCreate((TaskFunction_t)KEY_Task, /* 任务入口函数 */
119.                        (const char*   )"KEY_Task", /* 任务名字 */
120.                        (uint16_t      )512, /* 任务栈大小 */
121.                        (void*         )NULL, /* 任务入口函数参数 */
122.                        (UBaseType_t   )3, /* 任务的优先级 */
123.                        (TaskHandle_t*  )&KEY_Task_Handle); /* 任务控制块指针 */
124.    if(pdPASS == xReturn)
125.        printf("创建KEY_Task任务成功!\n");
126.
127.    vTaskDelete(AppTaskCreate_Handle); /* 删除AppTaskCreate任务
128.
129.    taskEXIT_CRITICAL(); /* 退出临界区
130. }
131.
132.
133.
134. /*****
135.  * @ 函数名 : LED_Task
136.  * @ 功能说明: LED_Task任务主体
137.  * @ 参数 :
138.  * @ 返回值 : 无
139.  *****/
140. static void LED_Task(void* parameter)
141. {
142.     EventBits_t r_event; /* 定义一个事件接收变量 */
143.     /* 任务都是一个无限循环，不能返回 */
144.     while (1)
145.     {
146.         /*****
147.         * 等待接收事件标志
148.         *
149.         * 如果xClearOnExit设置为pdTRUE，那么在xEventGroupWaitBits()返回之前，
150.         * 如果满足等待条件（如果函数返回的原因不是超时），那么在事件组中设置
151.         * 的uxBitsToWaitFor中的任何位都将被清除。
152.         * 如果xClearOnExit设置为pdFALSE，
153.         * 则在调用xEventGroupWaitBits()时，不会更改事件组中设置的位。
154.         *
155.         * xWaitForAllBits如果xWaitForAllBits设置为pdTRUE，则当uxBitsToWaitFor中
156.         * 的所有位都设置或指定的块时间到期时，xEventGroupWaitBits()才返回。
157.         * 如果xWaitForAllBits设置为pdFALSE，则当设置uxBitsToWaitFor中设置的任何

```

```

158.     * 一个位置1 或指定的块时间到期时，xEventGroupWaitBits() 都会返回。
159.     * 阻塞时间由xTicksToWait参数指定。
160.     *****/
161.     r_event = xEventGroupWaitBits(Event_Handle, /* 事件对象句柄 */
162.                                   KEY1_EVENT|KEY2_EVENT, /* 接收线程感兴趣的事件 */
163.                                   pdTRUE, /* 退出时清除事件位 */
164.                                   pdTRUE, /* 满足感兴趣的所有事件 */
165.                                   portMAX_DELAY); /* 指定超时事件，一直等 */
166.
167.     if((r_event & (KEY1_EVENT|KEY2_EVENT)) == (KEY1_EVENT|KEY2_EVENT))
168.     {
169.         /* 如果接收完成并且正确 */
170.         printf ( "KEY1与KEY2都按下\n");
171.         LED1_TOGGLE; //LED1 反转
172.     }
173.     else
174.         printf ( "事件错误! \n");
175. }
176. }
177.
178. /*****
179.  * @ 函数名 : KEY_Task
180.  * @ 功能说明: KEY_Task任务主体
181.  * @ 参数 :
182.  * @ 返回值 : 无
183.  *****/
184. static void KEY_Task(void* parameter)
185. {
186.     /* 任务都是一个无限循环，不能返回 */
187.     while (1)
188.     {
189.         if( Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN) == KEY_ON ) //如果KEY2被单击
190.         {
191.             printf ( "KEY1被按下\n");
192.             /* 触发一个事件1 */
193.             xEventGroupSetBits(Event_Handle,KEY1_EVENT);
194.         }
195.
196.         if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN) == KEY_ON ) //如果KEY2被单击
197.         {
198.             printf ( "KEY2被按下\n");
199.             /* 触发一个事件2 */
200.             xEventGroupSetBits(Event_Handle,KEY2_EVENT);
201.         }
202.         vTaskDelay(20); //每20ms扫描一次
203.     }
204. }
205.
206. /*****
207.  * @ 函数名 : BSP_Init
208.  * @ 功能说明: 板级外设初始化，所有板子上的初始化均可放在这个函数里面
209.  * @ 参数 :
210.  * @ 返回值 : 无
211.  *****/
212. static void BSP_Init(void)
213. {
214.     /*
215.     * STM32中断优先级分组为4，即4bit都用来表示抢占优先级，范围为：0~15
216.     * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，
217.     * 都统一用这个优先级分组，千万不要再分组，切忌。
218.     */
219.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
220.
221.     /* LED 初始化 */
222.     LED_GPIO_Config();
223.
224.     /* 串口初始化 */
225.     USART_Config();
226.
227.     /* 按键初始化 */
228.     Key_GPIO_Config();
229. }
230. }
231.
232. /*****END OF FILE*****/

```



七、实验现象

程序编译好，用 USB 线连接电脑和开发板的 USB 接口（对应丝印为 USB 转串口），用 仿真器把配套程序下载到 STM32 开发板，在电脑上打开串口调试助手，然后复位开发板就可以在调试助手中看到串口的打印信息，按下开发版的 KEY1 按键发送事件 1，按下 KEY2 按键发送事件 2；我们按下 KEY1 与 KEY2 试试，在串口调试助手中可以看到运行结果，并且当事件 1 与事件 2 都发生的时候，开发板的 LED 会进行翻转。

